Viscan: Dental Finding Detection in Panoramic X-rays

Technical report for the AI Engineer (Round 2) take-home assessment. 2D OPG track, plus a bonus 3D CBCT track.

Candidate: Surendra · **Date:** 26 October 2026 · **Code:** `main` of github.com/surendra902/viscan-opg at this commit · **Experiments:** executed at `1ac1080` · **Weights:** `best_model.pth`, 173,466,789 bytes, SHA-256 `e0b49b12f48fc71ec1f583db08f107650659571e7c2b9cbd454dbfed13b46a1f` · **Demo:** huggingface.co/spaces/surendra08/viscan-opg · **Video:** not recorded yet

> **Clinical disclaimer:** AI-assisted findings for dentist review. Not a diagnosis. This build is not clinically ready and has not been validated on clinical data.

1. Summary

Viscan takes a panoramic dental X-ray and returns boxes for four findings — impacted tooth, caries, periapical lesion and deep caries — each with an estimated FDI tooth number, plus an annotated image and JSON. The same endpoint accepts a 3D CBCT scan (§8). The detector is Faster R-CNN ResNet-50 FPN V2 (COCO_V1), 43.3 M parameters, fine-tuned for 12 epochs on DENTEX 2023.

On 50 held-out images (182 boxes) it scores **mAP@50 0.496** and **mAP@50:95 0.329**; macro-F1 across the four classes is **0.402** at the shipped threshold of 0.70. Impacted teeth are usable (F1 0.81); the other three classes are weak (0.37, 0.26, 0.17), so treat the output as a highlighter, not a filter. CPU latency on this laptop moves more than any model change I measured — 8–11 s per forward pass and 4–19 s per full request depending on the session — so the only latency figure I treat as a property of a method is the paired TTA ratio, 1.94× (§5, §6).

The best measured change is horizontal-flip TTA: mAP@50 rises from 0.496 to 0.528, at a CPU cost I now measure in pairs rather than guess (§6: 1.94× the single pass), while F1 at its own threshold is 0.399 against 0.402. I ship the single pass. §6 lists every configuration I ran.

2. Intended use and safety

A research prototype for a dentist-facing review workflow: it highlights candidate regions, does not diagnose, and its confidence score ranks findings rather than giving the probability of disease. `tooth_fdi` is a position-based estimate, labelled as one in the JSON (`tooth_fdi_basis`, `tooth_numbering`) and on the drawn box (`(est)`), and it stays `null` when the fitted table is missing instead of being guessed; §5 has its measured accuracy. Patient tags are stripped from DICOM. The 3D track gives a straight-line millimetre gap between two segmented surfaces, not a traced nerve or a risk score; without 3D weights raw uploads get HTTP 503 and demo data is never substituted. The fine-tuned 2D weights are non-commercial (§9).

3. Data card

Dataset. DENTEX (MICCAI 2023, CC BY-NC-SA 4.0) is the only public dataset with expert boxes for all four findings, so it is the only one I used; Zhou et al. (2024) merges caries classes and has no impaction label, and Tufts is restricted. Source: Hugging Face `ibrahimhamamci/DENTEX`, revision `7b27ccc8e342dcb774f69adc6ca5e6c09fefce93`.

Split. DENTEX has 705 labelled training images and 50 official validation images with labels; I did not use the 250 challenge-test images. My working set is 755 labelled images and 3,711 boxes, split with seed 42. The test set is the official 50-image validation set.

Split	Images	Boxes	Impacted	Caries	Periapical	Deep caries	Role
Train	635	3,190	545	2,004	144	497	Teaching the model
Validation	70	339	59	185	14	81	Epoch and threshold selection
Test	50	182	40	101	9	32	Final scores only

The test set is small — 9 periapical and 32 deep-caries boxes — so a handful of images can move a per-class score.

Leakage. The 70 validation images were drawn from the 705-image training pool with a seeded shuffle. I checked in code that no filename appears in two splits. DENTEX has no patient IDs, so I cannot rule out one patient appearing in two splits.

Preparation. Boxes `[x, y, width, height]` are clipped to the image and converted to `[x_min, y_min, x_max, y_max]`; no classes were merged. Images are resized to fit 800 × 1333, about 0.47× on average. DICOM files apply the modality LUT then the VOI LUT; MONOCHROME1 images are flipped. An input gate rejects blank, corrupt, colour, portrait and screenshot-like files before inference.

Label depth. Each DENTEX box carries three levels — quadrant (`category_id_1`), tooth position (`category_id_2`), diagnosis (`category_id_3`) — and the diagnosis level has exactly the four classes above. Restorations, root-canal fillings, missing teeth and bone loss are unlabelled here, so they are neither in the model nor reported; the two axes DENTEX does label feed `tooth_fdi` (§5). Zenodo record 15487430 carries them (1,808 OPG JPEGs, 14 YOLO classes) but adds a non-commercial line to its CC BY 4.0 licence, and using it is a retrain with a published class map, not a config change (§9).

Class imbalance. I did not use class weights or oversampling. The only imbalance-aware choices are per-class reporting and a single global threshold chosen on validation macro-F1; per-class thresholds and a rebalanced loss were not run. This is a limitation, not a solution (§9). Caries outnumbers periapical lesions about 14 to 1 in training. The raw data is not in the repository; `prepare_data.py` downloads and prepares it.

4. Model and training

I compared two options and trained the first: Faster R-CNN R50-FPN V2 (43.3 M parameters, BSD-3-Clause code and COCO starting weights) against Ultralytics YOLO11n (about 2.6 M, AGPL-3.0, not trained). DENTEX's reference run is a Faster R-CNN, so this is a like-for-like check, and the permissive licence avoids YOLO's AGPL obligation; the cost is a larger, slower model. There is no measured architecture comparison.

Training setup (`configs/baseline.yaml`): fine-tuned from COCO-pretrained weights; seed 42 with deterministic mode; SGD (learning rate 0.005, momentum 0.9, weight decay 0.0005); cosine schedule over 12 epochs; batch size 2; mixed precision on a Tesla T4; input 800 × 1333. The best checkpoint is epoch 8 (validation mAP@50 0.570, mAP@50:95 0.360). Low-scoring boxes are kept internally so the precision-recall curves are complete; the 0.70 floor is applied afterwards.

```
python prepare_data.py data/dentex
python train.py --config configs/baseline.yaml
python evaluate.py --weights checkpoints/best_model.pth --data_dir data/dentex
```

5. Results (50 held-out images, 182 boxes)

Overall: mAP@50 0.496, mAP@50:95 0.329, mAP@75 0.383, AR@100 0.683, macro-F1 0.402 at 0.70. Boxes are usually in the right place but not drawn tightly (AP50 0.496 vs AP75 0.383).

Class	Test boxes	AP@50	AP@50:95	Precision	Recall	F1
`impacted_tooth`	40	0.833	0.541	76.1%	87.5%	0.81
`caries`	101	0.358	0.244	27.7%	53.5%	0.37
`deep_caries`	32	0.457	0.302	42.9%	18.8%	0.26
`periapical_lesion`	9	0.336	0.230	33.3%	11.1%	0.17

The clearest result is the 0.81-to-0.17 F1 range: impacted teeth are usable, the other three classes are not; the mean is (0.814 + 0.365 + 0.261 + 0.167) / 4 = 0.402. Matching is class-aware at IoU ≥ 0.50, so a box in the right place with the wrong class counts against both classes.

Of 182 true boxes, 96 were found, 66 missed and 20 swapped class; the 142 background false alarms are 124 caries (§7).

Operating point. I swept confidence 0.10–0.90 on the 70 validation images and picked 0.70, where validation macro-F1 peaked at 0.556, then read test once. Test F1 is 0.402: 50 images is small and one global setting suits rare classes badly.

Tooth numbers. `fdi.py` fits an arch-geometry table on the 3,190 train boxes — midline threshold, arch-level threshold, one centroid per tooth position per quadrant — and reads a two-digit FDI number off each box centre. Its one knob, tuned on a one-in-five image holdout inside train, came out `v_weight = 0.0`: horizontal position alone wins. On the 339 validation ground-truth boxes, measured and not fitted, every box gets a number: quadrant 0.976, tooth digit 0.549, exact FDI 0.543. That is a ceiling — a detected box inherits the detector's errors, neighbours are the common miss, and an impacted tooth that never erupted has no correct number. `scripts/dentex_fdi.py` rebuilds the table from 1.2 MB range-read out of the 10.9 GB archive; `tests/test_fdi.py` pins the FDI convention and the train-fit / validation-measure rule.

CPU speed. Absolute latency does not hold still enough to carry the differences I am reporting. Model-only on the five bundled samples, three runs each: 11.1 s mean on two threads (median 11.3 s) and 8.3 s on ten cores (median 7.6 s) — `assets/eval/cpu_latency_5_t2.json`, `cpu_latency_5_t10.json`. The same two-thread setting measured 15.0 s half an hour earlier, and a 10-image held-out set measured 9.3 s and 5.8 s on two and ten

threads in an earlier session (`assets/eval/cpu_latency_10_t*.json`). A full request — decode, gate, model, drawing, two PNG writes — took 4.1–6.6 s against a warm server in one session and 9.4–19.3 s in another, one image 9.4 s then 16.6 s back to back (`assets/eval/live_request_latency.json`); each `samples/*_report.json` keeps the `inference_ms` of the request that wrote it. Peak process memory 2.9 GB. The paired §6 ratio (1.94×) is the number that survives, because both passes are timed in one process. Even the fast session is too slow for a clinic queue, so production would load once, serve one request at a time, report readiness on `/health` and fail closed with 503 rather than invent a result.

6. The measured comparison

P0–P7 come from one executed Kaggle T4 session at repo commit `1ac1080`; the executed notebook is `kaggle_beat_benchmark_notebook.ipynb` (17/17 code cells with outputs). It first reproduced the shipped baseline (AP 0.3293, AP50 0.4961, AP75 0.3830, AR@100 0.6834, F1 0.4016) with drift ≤ 0.0004. Thresholds and pipelines were selected on validation; the 50-image test set was read once per pipeline.

Pipeline	Test AP	Test AP50	Δ AP50 vs P0	Test F1 at its τ
P0 baseline (shipped)	0.3293	0.4961	—	0.4016 at 0.70
P1 same weights + hflip TTA	0.3446	0.5281	+0.0320	0.3988 at 0.80
P2 full view (hflip TTA) + native-scale tiles	0.2189	0.3557	−0.1404	0.2896 at 0.35
P3 P2 + RetinaNet, WBF	0.1790	0.3015	−0.1946	0.2661 at 0.30
P4 P2 + crop checker, re-score	0.2211	0.3589	−0.1372	0.3543 at 0.40
P5 P3 + crop checker, re-score	0.1936	0.3168	−0.1793	0.3172 at 0.40
P6 P2 + crop checker, re-label	0.1700	0.2715	−0.2246	0.3184 at 0.25
P7 P3 + crop checker, re-label	0.1437	0.2356	−0.2605	0.2320 at 0.30

Only P1 beat the baseline on detection score, and not at the setting I would run: +0.032 AP50 and +0.015 AP for twice the forward passes, with F1 at its own validation-selected threshold slightly lower. Its CPU cost is now measured: `scripts/benchmark_cpu.py --tta both` times the plain pass and the mirror pass as paired runs of the same image in one process, and the mirror pass takes 1.936× the single pass (per-image median 1.979×, range 1.618–2.121×; `assets/eval/cpu_latency_tta_paired_t2.json`). Comparing two separate processes had suggested only 1.44×, which is why the two passes are timed in pairs. I ship P0.

The rest lost 0.137–0.261 AP50. P2 tiles an 800 px crop at about twice the object size the model learned and overflowed the 300-box cap. P3's RetinaNet was undertrained, not shown to be worse: validation mAP@50 was 0.0783 at epoch 12/12 and still rising. The crop checker helped slightly as a re-scorer (+0.003 AP50 over P2) and hurt as a re-labeller (0.272); its 92.1% is training accuracy on its own crop bank, not held-out.

A 960×1600 run, from the executed training notebook `kaggle_notebook.ipynb` rather than this ladder, drew tighter boxes (AP75 0.410 vs 0.383) and found impacted teeth better (AP50 0.915 vs 0.833) but lost overall (AP50 0.493, AP 0.313) and on deep caries (0.365); its batch size also changed, so resolution is not isolated. Per-class thresholds were not run.

7. Failure cases

At 0.70 the model made 142 false alarms in 44 of 50 images, missed 66 boxes in 33 images and swapped 20 classes. The five cases were exported from Cell 9 of `kaggle_notebook.ipynb`, at most one per image; the explanations are hypotheses, untested.

#	Image	What happened	Likely reason
1	`val_4`	False alarm: impacted tooth, score 0.99, IoU 0.00	Confident about back-of-jaw anatomy the annotators did not mark; unlabelled third molars are plausible.
2	`val_38`	Missed a periapical lesion	Rare class (144 training boxes); only 1 of 9 was found at this setting.
3	`val_3`	Deep caries called caries, score 0.97	Box is right, severity is wrong; 14 of 20 swaps are this pair.
4	`val_33`	Missed a caries	Caries recall is 53.5% and the scan is shrunk 0.47×, so a small gap-between-teeth cavity is the expected casualty.
5	`val_13`	False alarm: impacted tooth, score 0.99, IoU 0.00	Same error as case 1; the repeat suggests it is the model's most confident mistake, not a one-off.

The five exported cases are the model's most confident errors; its most frequent error, caries false alarms, shows only in the totals.

8. Bonus: 3D CBCT

The same `/analyze` endpoint accepts `.nii`, `.nii.gz` and ZIP archives of a CBCT scan. I use DentalSegmentator (Dot et al., 2024) as-is, without training, to segment five structures: upper jaw, mandible, upper teeth, lower teeth and mandibular canal. My work is loading, patient-tag removal, 60 mm^3 island cleanup, distance measurement, mesh export and browser viewer.

The finding is the smallest straight-line gap between the lower teeth and the nerve canal, per half of the volume: 0.71 mm and 1.58 mm on the Post-surgery demo scan, 2.12 mm and 1.50 mm on Pre. The halves are a voxel midpoint split, not the patient's real left and right, and the gap is not a traced nerve or a risk score. A full Post run on a Tesla T4 takes 131.4 s of inference over 252 tiles, 5.8 s of post-processing and 0.9 s of meshing, peaking at 4,936 MB of GPU memory. The Post labels are the published SlicerDentalSegmentator reference mask, matched at mean Dice 0.964 (canal 0.897); the Pre labels have no reference mask, so no Dice is reported. The two scans are not claimed to be the same patient. Pre-segmented bundles skip the network (about 9.6 s Post and 10.4 s Pre on CPU); the Pre bundle's `run_metrics.json` is not backed by an executed notebook, so I do not report it.

The free Space has no 3D model, so raw uploads there return HTTP 503 and only the two bundled scans work; demo data is never substituted for a failed inference. The production plan — queued jobs, warm GPU workers, compressed meshes — is in README §7, not a benchmark.

9. Licensing, limits and next work

Code is BSD-3-Clause. DENTEX and the fine-tuned `best_model.pth` are CC BY-NC-SA 4.0, so the weights are non-commercial and would need retraining on cleared data. The DENTEX images bundled in `samples/`, and the annotated figures drawn from them, carry the same CC BY-NC-SA 4.0 terms: BSD-3-Clause covers the code, not those pictures. DentalSegmentator weights are CC BY 4.0; the Slicer demo scans are BSD-style; runtime libraries are MIT, BSD, Apache-2.0 or HPND. YOLO11n (AGPL-3.0) was not used. The code could ship commercially; the trained 2D weights could not.

The rare classes are the main limit: periapical recall 11.1% and deep-caries recall 18.8% at 0.70, and 14 of 32 deep boxes are called shallow. Caries precision is 27.7%. There is no class balancing and only one global threshold. The tooth number is a geometry estimate right on 0.543 of validation boxes, not an anatomical one. The model labels only the four findings DENTEX annotates, was trained on one dataset, and 2D projection overlap cannot separate anatomy from lesions; the input gate uses hand-written rules, so unusual X-rays can be wrongly rejected.

With two more weeks: per-class confidence settings tuned on validation (1–2 days, not measured); class-balanced loss and oversampling for periapical and deep caries (about a week); RetinaNet with a schedule suited to 635 images (2–3 GPU days); ONNX export with INT8 quantisation (3–5 days, target unmeasured); a tooth-detection stage so `tooth_fdi` comes from the anatomy in the image rather than from a box centre (about two weeks); and a second annotated dataset for the classes DENTEX does not label — restorations, root-canal fillings, missing teeth, bone loss — which is a retrain with a published class map, not a config change. Mirror averaging is the cheap detection extra (+0.032 AP50, measured at 1.94× the single-pass CPU time in §6), which is why I still ship the single pass.

10. Reproduce and AI disclosure

```
docker compose up              # or: python app.py
python scripts/smoke_test.py   # after the server is up
```

The first start fetches and SHA-256-verifies the 173 MB checkpoint. The Python path, the smoke test, the tooth-number fit and the CPU benchmarks (10 held-out images, the 5 bundled samples at two and ten threads, and the live-request run) were done on Windows with Python 3.11; the Docker image was not built on this machine. Training and evaluation commands are in §4. AI tools used are listed in AI_DISCLOSURE.md.

References. Hamamci et al., DENTEX, MICCAI 2023. Dot et al., DentalSegmentator, *Journal of Dentistry*, 2024 (Zenodo 10.5281/zenodo.10829675). Isensee et al., nnU-Net, *Nature Methods*, 2021. Akyon et al., SAHI, arXiv:2202.06934. Solovyev et al., Weighted Boxes Fusion, 2021. Lin et al., COCO, ECCV 2014 (`pycocotools`).